

基于验证码识别的CNN——机器学习基础

版本：2026-05-13

目标读者：

- 网络安全安全学院学生 `pwnnofans`：有一定 Python / PyTorch / CNN 实验经验，希望从定性过程上建立机器学习整体框架。
- 数学院学生 `屠狗`：计算机基础可以为零，但具备数学理解能力，希望看到清楚的数学对象、公式和直觉。

本文目标：

不是列举大量模型，而是先回答一个最基本的问题：

机器学习到底在学习什么？它如何把现实任务变成数学问题？

1. 机器学习到底要解决什么？

假设现在有一个任务：

- 输入：一张验证码图片
- 输出：验证码中的 4 个字符

例如：

- 图片 $\rightarrow$ `a7k2`

如果让人来识别，人会看：

- 字符的轮廓
- 笔画方向
- 黑白区域
- 字符之间的大致位置
- 局部形状差异

但计算机不能直接理解“笔画”“轮廓”“字符”这些概念。

它看到的只是像素矩阵：

$$X \in \mathbb{R}^{H \times W}$$

如果是 RGB 彩色图片，则可以写成：

$$X \in \mathbb{R}^{C \times H \times W}$$

其中：

C = 通道数，即RGB三色，这里为3， H = 高度，比如1080px， W = 宽度，比如1920px

首先我们进行数学建模

- **样本**：一张验证码图片
- **特征**：图片中的像素值，或者由模型进一步提取出的边缘、笔画、局部形状等特征

数学上可以把一个样本记为：

$$x$$

如果有很多验证码图片，可以记为：

$$x_1, x_2, \dots, x_n$$

这就是我们样本的**训练集**。

接着，我们还需要告诉机器正确答案是什么。比如这张图的真实验证码是：`a7k2`

这就是**标签**。

数学上可以记为：

$$y = (y_1, y_2, y_3, y_4)$$

其中：

$$y_i \in \{0, 1, 2, \dots, 35\}$$

表示第 i 个字符属于 `26 + 10 = 36` 个类别中的哪一类。

于是训练数据可以写成：

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

然后我们希望机器学到一个“黑箱函数”：验证码图片 $\rightarrow$ 字符序列

数学上就是：

$$\hat{y} = f_{\theta}(x)$$

其中：

$$x = \text{输入验证码图片}$$

这个函数就是我们的**模型**。

但是一开始，这个模型肯定不准。

而找到这个 $f_{\theta}(x)$ 的过程就是机器学习的过程。初始时随便给 θ 一个值， $f_{\theta}(x)$ 的输出就是随机的

比如真实答案是：`a7k2`，模型可能预测成：`1zh2`。

为了将模型的输出拉回正轨，这时就需要定义：

预测结果和真实答案到底差了多少？

这就是损失函数。

数学上可以写成：

$$L(\hat{y}, y)$$

对于整个训练集，平均损失可以写成：

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(f_{\theta}(x_i), y_i)$$

如果差距大，就说明模型当前的内部参数不合适。于是我们希望机器自动修改这个函数内部的参数，让它下次更可能输出正确结果。

这里就引出优化和梯度下降：

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

其中：

$$\eta = \text{学习率}$$

定性理解就是：

预测错误 → 损失变大 → 计算应该如何调整参数 → 更新模型 → 下一次预测更接近真实标签。

训练很多张验证码之后，模型逐渐学会：

- 某些局部像素组合更像数字 7
- 某些弯曲笔画像字母 k
- 某些位置大概率对应第 1、2、3、4 个字符

而这里会引入两个问题

- 如果一直拿这一张图片去进行上述的迭代过程，那么最终的黑箱的输出就会变成固定的 `a7k2`。
 - 这就是**过拟合**。由于数据过于局限，没法让AI学习到更加广泛的特征，局限于训练集的数据，只能预测训练集的数据。
- 我们真正关心的不是它能不能记住训练集中见过的验证码，而是：换一张它从没见过的新验证码，它还能不能识别？
 - 这就引出**泛化**。

因此，从这个任务可以自然推导出机器学习的基本问题链条：

我要解决什么任务？

- 什么东西算一个样本？
- 样本如何变成数字特征？
- 正确答案如何定义成标签？
- 用什么模型函数从输入映射到输出？
- 如何衡量预测和标签的差距？
- 如何根据差距修改模型参数？
- 如何判断模型不是死记硬背，而是真的能处理新样本？

有了这个例子，就可以自然过渡到下面这段更正式的定义：

机器学习是现代 AI 的核心方法之一。

它的基本思想是：给机器大量样本，让机器从样本中学习规律，并用这个规律处理新的样本。

不同模型的本质区别，在于它们对研究对象的数据结构作出了不同假设。例如，有的模型假设特征和结果之间近似线性相关，有的模型假设图像中局部模式很重要，有的模型假设序列中的前后关系很重要。

在监督学习中，最典型的任务是分类和回归。

分类是把样本划入某个类别，回归是预测一个连续数值。

无论分类还是回归，本质上都可以看成学习一个函数：输入样本特征，输出预测结果。

为了让计算机学习，我们需要把现实对象转化为数字形式，也就是定义样本和特征；还需要定义标签，也就是我们希望模型预测的目标。

模型根据输入给出预测结果，再通过损失函数衡量预测和真实标签之间的差距。如果差距较大，就通过梯度下降等优化方法调整模型参数，使下一次预测更接近目标。

因此，机器学习的核心过程可以理解为：定义数据表示，选择模型结构，定义损失函数，通过优化算法寻找一组参数，使模型不仅能拟合训练数据，还能在未见过的数据上保持较好表现。

2. 机器学习的基本对象

通过以上引入，可以理解的机器学习最基本的对象有：

- 样本 `sample`
- 特征 `feature`
- 标签 `label`
- 模型 `model`
- 参数 `parameter`
- 预测 `prediction`
- 损失 `loss`
- 优化 `optimization`
- 泛化 `generalization`

3. 将一次训练过程写成可以计算的代数形式

上面我们已经得到机器学习的基本流程：

$$x \rightarrow f_{\theta}(x) \rightarrow \hat{y} \rightarrow L(\hat{y}, y) \rightarrow \theta \text{ 更新}$$

接下来我们用一个最简单的模型，把这个过程写成完全可以计算的代数形式。

即假设验证码的最终数字与图片中的每一个像素存在线性关系，可以使用线性回归预测。

需要注意：

下面使用线性模型只是为了展示“机器学习如何训练”的基本代数过程。

对于验证码识别这种图像任务，简单线性模型通常无法很好解决，因为它无法自然表达笔画、局部形状、位置变化、字符结构等复杂特征。

但线性模型非常适合作为第一个数学例子，因为它的输入、输出、损失、梯度和参数更新都可以直接算出来。

3.1 将图片样本变成向量

假设一张验证码图片原本是二维像素矩阵：

$$X \in \mathbb{R}^{H \times W}$$

为了使用最简单的线性模型，我们先把它拉平成一个向量：

$$x \in \mathbb{R}^d$$

其中：

$$d = H \times W$$

如果原图是：

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots \\ x_{21} & x_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

拉平成向量后可以写成：

$$x = [x_1, x_2, \dots, x_d]^T$$

这里的每一个 x_j 都可以理解为某个像素位置的数值。

例如灰度图中：

$$x_j \in [0, 1]$$

表示该像素从黑到白的强度。

3.2 定义一个最简单的线性模型

先考虑最简单的情况：

模型只输出一个数字。

例如这个数字可以表示：

- 某个字符类别的打分；
- 某个样本属于某类的程度；
- 某个连续预测值。

我们定义线性模型：

$$\hat{y} = f_{\theta}(x) = w^T x + b$$

展开就是：

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

其中：

$$w = [w_1, w_2, \dots, w_d]^T$$

是权重向量， b 是偏置项。

这里的参数就是：

$$\theta = (w, b)$$

所以这个模型的含义是：

每一个特征 x_j 都会乘上一个权重 w_j ，然后所有结果相加，再加上偏置 b ，得到模型预测值 $\hat{y}$ 。

3.3 线性模型的直觉

如果 w_j 很大，说明第 j 个特征对输出影响很大。

如果 w_j 接近 0，说明第 j 个特征几乎不影响输出。

如果 w_j 是负数，说明第 j 个特征越大，输出反而越小。

因此，线性模型可以理解为：

给每个输入特征分配一个重要性权重，然后加权求和。

在验证码图片中，如果把每个像素当成一个特征，那么线性模型实际上是在说：

每个像素位置对最终预测都有一个固定重要性。

这也是它的局限。

因为真实验证码识别并不是某几个固定像素决定的，而是由：

- 局部笔画组合；
- 字符轮廓；
- 相邻像素结构；
- 字符位置变化；
- 字符之间的空间关系；

共同决定的。

简单线性模型很难表达这些结构。

3.4 定义损失函数：预测和真实值差多少

为了让代数过程最清楚，我们先使用最简单的均方误差损失。

对于一个样本 (x, y) ，模型预测为：

$$\hat{y} = w^T x + b$$

真实答案为：

$$y$$

定义损失函数：

$$L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

这里加上 $\frac{1}{2}$ 是为了后面求导更方便。

因为：

$$\frac{d}{dz} \frac{1}{2} z^2 = z$$

所以 $\frac{1}{2}$ 不改变损失函数的本质，只是让公式更简洁。

3.5 一次训练的目标

我们希望损失越小越好。

也就是希望：

$$\hat{y} \approx y$$

即：

$$w^T x + b \approx y$$

如果当前模型预测错了，就说明当前参数 w, b 不够好。

于是我们要调整参数：

$$w_1, w_2, \dots, w_d, b$$

让下一次预测更接近真实值。

3.6 计算损失对参数的影响

因为：

$$\hat{y} = w^T x + b$$

令预测误差为：

$$e = \hat{y} - y$$

则：

$$L = \frac{1}{2}e^2$$

现在我们要求损失对每个参数的梯度。

首先：

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = e$$

又因为：

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b$$

所以：

$$\frac{\partial \hat{y}}{\partial w_j} = x_j$$

根据链式法则：

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_j} = (\hat{y} - y)x_j$$

也就是：

$$\frac{\partial L}{\partial w_j} = ex_j$$

对于偏置项：

$$\frac{\partial \hat{y}}{\partial b} = 1$$

所以：

$$\frac{\partial L}{\partial b} = \hat{y} - y = e$$

3.7 梯度的向量形式

把所有 w_j 合在一起，可以写成：

$$\nabla_w L = (\hat{y} - y)x$$

也就是：

$$\nabla_w L = ex$$

偏置项的梯度是：

$$\nabla_b L = e$$

所以一次样本上的梯度可以写成：

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

这说明：

参数更新方向由两个东西决定：

第一，模型当前错了多少，即 $\hat{y} - y$ ；

第二，当前输入特征是多少，即 x 。

如果某个特征 x_j 很大，并且预测误差也很大，那么对应的权重 w_j 就会被更明显地调整。

3.8 使用梯度下降更新参数

梯度下降的更新公式是：

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

对于线性模型，就是：

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}$$

代入上面的梯度：

$$w_{t+1} = w_t - \eta(\hat{y} - y)x$$

其中：

$$\eta$$

是学习率。

定性理解：

如果预测值比真实值大，说明模型输出偏高，参数要往降低输出的方向调。

如果预测值比真实值小，说明模型输出偏低，参数要往提高输出的方向调。

调整多少，由误差大小和学习率共同决定。

3.9 一个完整的数值例子

为了让这个过程完全可计算，我们假设一个极简样本只有两个特征：

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

真实标签是：

$$y = 1$$

初始模型参数为：

$$w = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$$

学习率：

$$\eta = 0.1$$

第一步：前向传播

模型预测：

$$\hat{y} = w^T x + b$$

代入数值：

$$\hat{y} = 0.1 \times 2 + 0.2 \times 1 + 0$$

所以模型预测为：

$$\hat{y} = 0.4$$

真实值是：

$$y = 1$$

说明模型预测偏低。

第二步：计算损失

损失函数：

$$L = \frac{1}{2}(\hat{y} - y)^2$$

代入：

$$L = \frac{1}{2}(0.4 - 1)^2$$

第三步：计算误差

$$e = \hat{y} - y$$

第四步：计算梯度

权重梯度：

$$\frac{\partial L}{\partial w} = ex$$

代入：

$$\frac{\partial L}{\partial w} = -0.6 \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

所以：

$$\frac{\partial L}{\partial w} = \begin{bmatrix} -1.2 \\ -0.6 \end{bmatrix}$$

偏置梯度：

$$\frac{\partial L}{\partial b} = e = -0.6$$

第五步：更新参数

更新权重：

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

代入：

$$w_{\text{new}} = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} - 0.1 \begin{bmatrix} -1.2 \\ -0.6 \end{bmatrix}$$

更新偏置：

$$b_{\text{new}} = b - \eta \frac{\partial L}{\partial b}$$

第六步：用新参数再预测一次

新参数：

$$w_{\text{new}} = \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix}$$

再次预测：

$$\hat{y}_{\text{new}} = 0.22 \times 2 + 0.26 \times 1 + 0.06$$

新预测值从：

0.4

变成：

0.76

更接近真实标签：

1

新的损失为：

$$L_{\text{new}} = \frac{1}{2}(0.76 - 1)^2$$

损失从：

0.18

下降到：

0.0288

这就展示了一次完整训练更新的代数过程。

3.10 这说明了什么？

这次训练过程可以总结为：

$$x \rightarrow \hat{y} = w^T x + b \rightarrow L = \frac{1}{2}(\hat{y} - y)^2 \rightarrow \nabla L \rightarrow w, b \text{ 更新}$$

也就是：

输入样本 → 模型预测 → 计算误差 → 计算梯度 → 修改参数 → 预测更接近真实标签

这就是机器学习训练过程的最小数学骨架。

3.11 如果有多个样本怎么办？

实际训练不可能只用一个样本。

假设训练集有 n 个样本：

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

线性模型对第 i 个样本的预测为：

$$\hat{y}_i = w^T x_i + b$$

单个样本损失为：

$$L_i = \frac{1}{2}(\hat{y}_i - y_i)^2$$

整个训练集的平均损失为：

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (w^T x_i + b - y_i)^2$$

训练目标就是：

$$\min_{w, b} J(w, b)$$

也就是：

找到一组 w, b ，让所有训练样本上的平均预测误差尽可能小。

3.12 多样本情况下的梯度

对于整个训练集：

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (w^T x_i + b - y_i)^2$$

令：

$$\hat{y}_i = w^T x_i + b$$

则：

$$\frac{\partial J}{\partial w} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

所以参数更新为：

$$w_{t+1} = w_t - \eta \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

这就是批量梯度下降的基本形式。

3.13 为什么实际训练通常使用一批子集样本 batch?

如果训练集非常大，每次都使用全部样本计算梯度，计算成本很高。

所以实际训练通常每次只取一小批样本：

$$B = \{(x_1, y_1), \dots, (x_m, y_m)\}$$

其中 m 是 batch size。

batch 上的平均损失为：

$$J_B(w, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (w^T x_i + b - y_i)^2$$

对应梯度为：

$$\frac{\partial J_B}{\partial w} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_i$$

然后用这个 batch 的梯度更新参数。

这就是常见的 mini-batch gradient descent。

直觉是：

不用每次看完整个训练集，而是每次看一小批样本，用这一小批样本估计整体下降方向。

PS：你能从数学的角度说明batch取什么值比较合理么？ @ 屠狗

3.14 为什么这个线性模型不能很好解决验证码识别？

上面的模型是：

$$\hat{y} = w^T x + b$$

它的含义是：

每个像素位置乘一个固定权重，然后全部加起来。

这个模型的问题是：**线性模型的权重是按照像素固定位置的**

它描述的是一张图片中哪几个固定位置的像素对于最终的预测结果起了关键作用。

同一个字符稍微平移几个像素，像素位置就变了。所以它甚至不能识别字符平移的情况。

而且从局部来讲，我们需要识别横平竖直才能够识别出字符轮廓。

这里直接将一张图片展开成了向量，他就很难捕捉原本图片中高度方向的关系。

从本质来说就是线性表示降维导致了无法识别。

下一次笔记的任务就是进行数学建模，如何实现验证码中的字符轮廓的表示。

PS：你能从数学的本质来说明为什么不行么？ @ 屠狗

3.15 分类任务中的线性模型

上面的例子用了均方误差，是经典的线性回归。

如果将验证码识别建模为分类任务，线性模型通常不会直接输出最终类别，而是先输出每个类别的分数。

假设有 C 个类别。

例如验证码中单个字符有：

$$C = 36$$

个类别。

那么模型可以输出：

$$z = Wx + b$$

其中：

$$W \in \mathbb{R}^{C \times d}, d = H \times W$$

这里的：

$$z = [z_1, z_2, \dots, z_C]^T$$

叫做 `logits`，也就是每个类别的**原始分数**。

3.16 从 logits 到概率：softmax

为了把分数变成概率，使用 `softmax`：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}$$

其中：

$$p_k$$

表示模型认为样本属于第 k 类的概率。

所有类别概率满足：

$$\sum_{k=1}^C p_k = 1$$

并且：

$$p_k \geq 0$$

所以：

$$p = [p_1, p_2, \dots, p_C]$$

可以看成是一个概率分布。

PS：你能从这个概率函数的性质来说明这个定义的合理性么，你能设计类似的概率映射函数么？@ 屠狗

3.17 分类任务中的交叉熵损失

PS：这个熵的定义大多都是指数形式，熵的应用也有很多，你能从一个比较合理方向解释这个定义的合理性么？@ 屠狗

如果真实类别是第 r 类，那么交叉熵损失为：

$$L = -\log p_r$$

也就是说：

模型给真实类别的概率越高，损失越小。

模型给真实类别的概率越低，损失越大。

如果用 one-hot 表示真实标签：

$$y = [y_1, y_2, \dots, y_C]$$

其中真实类别对应位置为 1，其余为 0。

交叉熵可以写成：

$$L(p, y) = -\sum_{k=1}^C y_k \log p_k$$

因为只有真实类别 r 的 $y_r = 1$ ，所以它等价于：

$$L = -\log p_r$$

3.18 多字符验证码的损失

验证码有 4 个字符：

$$y = (y_1, y_2, y_3, y_4)$$

每个字符都是一个 36 分类问题：

$$y_i \in \{0, 1, 2, \dots, 35\}$$

模型需要对每个位置输出一个概率分布：

$$p^{(1)}, p^{(2)}, p^{(3)}, p^{(4)}$$

其中：

$$p^{(i)} \in \mathbb{R}^{36}$$

表示第 i 个字符位置上的 36 类概率。

如果第 i 个位置的真实类别是 y_i ，则该位置的损失是：

$$L_i = -\log p_{y_i}^{(i)}$$

整个验证码的损失可以取平均：

$$L = \frac{1}{4} \sum_{i=1}^4 L_i$$

也就是：

$$L = -\frac{1}{4} \sum_{i=1}^4 \log p_{y_i}^{(i)}$$

这和你前面验证码实验中“逐字符计算交叉熵，再对位置取平均”的思想是一致的。

3.19 这一节的核心总结

这一节展示了机器学习训练的最小代数形式。

对于最简单的线性模型：

$$\hat{y} = w^T x + b$$

对于均方误差损失：

$$L = \frac{1}{2}(\hat{y} - y)^2$$

梯度为：

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

参数更新为：

$$w_{t+1} = w_t - \eta(\hat{y} - y)x$$

所以机器学习训练可以通俗理解为：

模型先根据当前参数做预测；
损失函数衡量预测错了多少；
梯度告诉我们每个参数对错误的贡献方向；
优化器根据梯度修改参数；
多次重复后，模型逐渐学到从输入到输出的映射关系。

但对于验证码识别这样的图像任务，简单线性模型能力不足。

它只能表达：

像素加权求和 → 输出

而验证码识别需要表达：

像素 → 局部边缘 → 笔画 → 字符结构 → 多字符序列

因此，后面才需要继续学习 CNN、Transformer 等更复杂模型。